



**Università
degli Studi
di Palermo**



Introduzione al Machine Learning

CORSO DI BIG DATA – MODULO MACHINE LEARNING PER I BIG DATA
a.a. 2024/2025

Prof. Roberto Pirrone

Sommario

- Generalità
- Assunzioni del ML
- Stimatori e stima MLE e MAP
- Tipologie dei compiti di apprendimento
- Utilizzo dei dati
- Capacità e generalizzazione
- Tecniche di addestramento

Generalità

- Il Machine Learning – Apprendimento Automatico in Italiano – si riferisce allo sviluppo di *programmi per computer che siano in grado di apprendere dai dati*
- In generale, il programma avrà a disposizione una esperienza E , rispetto a una classe di compiti T e una misura di performance P
- *L'apprendimento implica che la performance P sulla classe di compiti T migliorerà utilizzando E*

Generalità

- Un algoritmo di apprendimento deve fornire la *stima statistica* di una funzione complessa che lega i dati alle uscite desiderate

Modello: caratterizzazione della famiglia di forme funzionali utilizzate per f e/o della tipologia di algoritmo impiegato

Parametri del modello: quantità che sono direttamente coinvolte nella forma funzionale di f e devono essere apprese dai dati

$$y = f(x, w; \theta)$$

Iperparametri: quantità che condizionano la corretta evoluzione dell'algoritmo, ma che devono essere fissate in fase di apprendimento; possono essere stimate usando i dati, ma con tecniche diverse dall'apprendimento vero e proprio

Assunzioni del ML

- Il Machine Learning è un processo di stima delle distribuzioni di probabilità sui dati a partire da quelli osservati dal modello

$$y = \mathcal{P}(\mathbf{x}, \mathbf{w}; \theta)$$

- Diverse tipologie di distribuzioni di probabilità che sottendono diverse tipologie di dato
 - Multidimensionali
 - Categorici
 - Serie discrete ...

Assunzioni del ML

- Si consideri un data set $\mathcal{D}$ di n campioni $\mathbf{x}^{(i)} \in \mathbb{R}^d$
- I campioni $\mathbf{x}^{(i)}$ sono da considerarsi le *manifestazioni casuali del fenomeno da essi descritto*, rappresentato dalla variabile aleatoria $\mathbf{x}$
- L'ipotesi base del machine learning sui campioni $\mathbf{x}^{(i)}$ è che essi siano statisticamente *indipendenti e identicamente distribuiti* (assunzione o ipotesi *i.i.d.*)

Stima statistica

- Uno stimatore o una *statistica* è una funzione dei dati in nostro possesso la quale cerca di fornire la *migliore predizione possibile* di una quantità o funzione cui siamo interessati
- La stima di $\boldsymbol{\vartheta}$ è puntuale, in funzione dei campioni $\boldsymbol{x}^{(i)}$

$$\hat{\boldsymbol{\theta}}_m = g(\boldsymbol{x}^{(1)}, \dots, \boldsymbol{x}^{(m)})$$

*Le stime sono esse
stesse variabili
aleatorie!!*

Stimatori polarizzati e non polarizzati

- La polarizzazione o *bias* di uno stimatore si misura come la differenza tra il suo valore atteso ed il valore vero della quantità da stimare

$$\text{bias}(\hat{\theta}_m) = \mathbb{E} [\hat{\theta}_m] - \theta$$

- Dipende dai dati che abbiamo per calcolare la stima

Stimatori polarizzati e non polarizzati

- La polarizzazione o *bias* di uno stimatore si misura come la differenza tra il suo valore atteso ed il valore vero della quantità da stimare

- Stimatore non polarizzato

$$\mathbb{E} \left[\hat{\theta}_m \right] = \theta$$

- Stimatore asintoticamente non polarizzato $\lim_{m \rightarrow \infty} \mathbb{E} \left[\hat{\theta}_m \right] = \theta$

Stimatori campionari di media e varianza

- Assumiamo che i campioni siano tratti da una distribuzione normale

$$\forall i, x^{(i)} \sim \mathcal{N}(x; \mu, \sigma)$$

$$\hat{\mu}_m = \frac{1}{m} \sum_{i=1}^m x^{(i)}$$

$$\hat{\sigma}_m^2 = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \hat{\mu}_m)^2 \quad \text{Polarizzato}$$

Stimatori campionari di media e varianza

- Assumiamo che i campioni siano tratti da una distribuzione normale

$$\forall i, x^{(i)} \sim \mathcal{N}(x; \mu, \sigma)$$

$$\tilde{\sigma}_m^2 = \frac{1}{m-1} \sum_{i=1}^m \left(x^{(i)} - \hat{\mu}_m \right)^2 \quad \text{Non polarizzato}$$

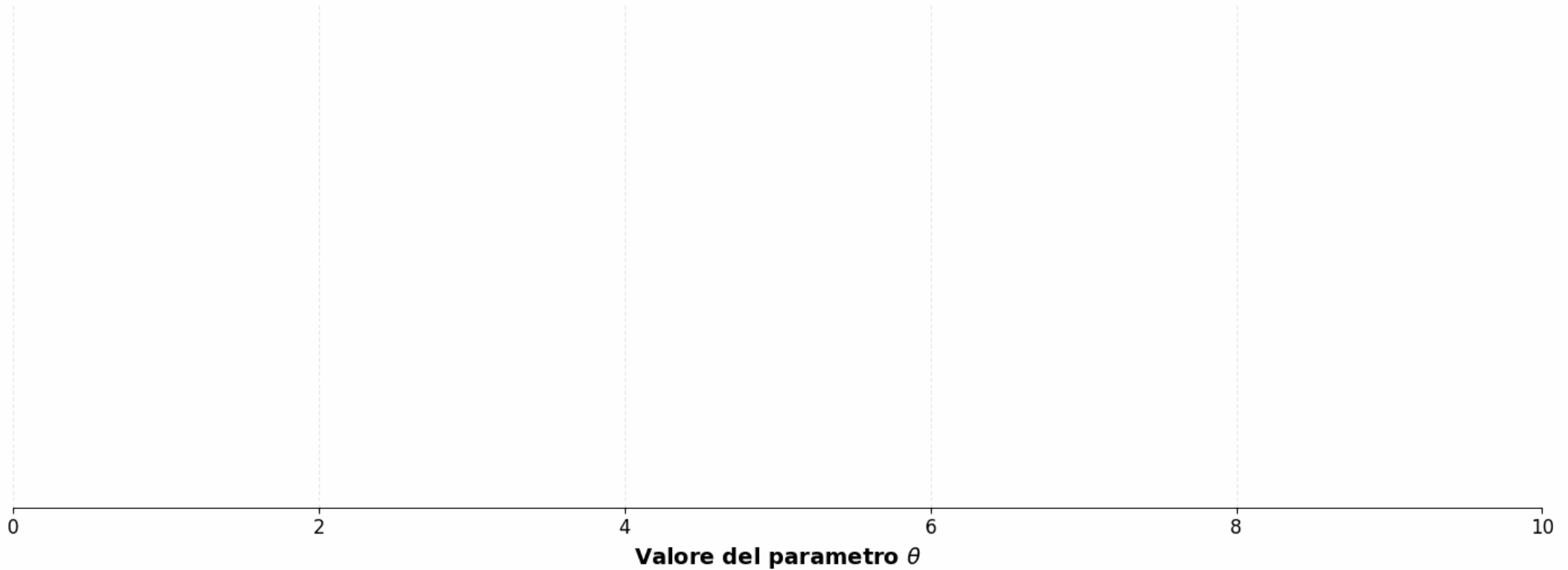
Varianza di uno stimatore

- È la misura utilizzata per quanto la stima sia stabile rispetto al variare dei campioni utilizzati
- La radice quadrata della varianza si definisce *standard error*

$$SE(\hat{\theta}_m) = \sqrt{\text{Var}(\hat{\theta}_m)}$$

Come si valuta la bontà di uno stimatore?

Bias e Varianza di uno Stimatore Statistico
Valore Vero del Parametro: $\theta = 5.0$



Errore quadratico medio (MSE)

- Lo stimatore ideale ha il minimo bias, meglio se sia non polarizzato, e la minima varianza
- Per giudicare la bontà di una stima in questo senso si utilizza l'*errore quadratico medio* (Mean Squared Error: MSE)

$$\text{MSE} = \mathbb{E} \left[(\hat{\theta}_m - \theta)^2 \right] =$$
$$\text{Bias}(\hat{\theta}_m)^2 + \text{Var}(\hat{\theta}_m)$$

Stima a Massima Verosimiglianza (MLE)

- Assumiamo di avere un dataset $\mathcal{D}$ dal quale vogliamo apprendere un modello descritto da un vettore di parametri $\mathbf{w}$
- Avremo appreso il miglior modello quando avremo massimizzato la «probabilità a posteriori» o *posterior* $P(\mathbf{w} | \mathcal{D})$
 - Ciò che possiamo stimare dopo aver osservato i dati

Stima a Massima Verosimiglianza (MLE)

- Dal teorema di Bayes possiamo trovare un'espressione per calcolare $P(\mathbf{w}|\mathcal{D})$ dalla definizione del nostro modello di Machine Learning che è invece $P(\mathcal{D}, \mathbf{w})$

$$\begin{aligned} P(\mathcal{D}, \mathbf{w}) &= P(\mathcal{D}|\mathbf{w})P(\mathbf{w}) = \\ &= P(\mathbf{w}|\mathcal{D})P(\mathcal{D}) \end{aligned}$$

$$P(\mathbf{w}|\mathcal{D}) = \frac{P(\mathcal{D}|\mathbf{w})P(\mathbf{w})}{P(\mathcal{D})}$$

Stima a Massima Verosimiglianza (MLE)

- Il teorema di Bayes correla il *posterior* con:
 - la conoscenza a priori o *prior* sul nostro modello: $P(\mathbf{w})$
 - la verosimiglianza o *likelihood* del nostro modello, espresso dalla probabilità di predire effettivamente $\mathcal{D}$ usando i parametri del modello $\mathbf{w}$: $P(\mathcal{D}|\mathbf{w})$

$$P(\mathbf{w}|\mathcal{D}) = \frac{P(\mathcal{D}|\mathbf{w})P(\mathbf{w})}{P(\mathcal{D})}$$

$$\text{posterior} \propto \text{likelihood} \times \text{prior}$$

Stima a Massima Verosimiglianza (MLE)

- La conoscenza a priori di $P(\mathcal{D})$ non è altro che la probabilità marginale espressa in termini del numeratore:

$$P(\mathcal{D}) = \int p(\mathcal{D}|\mathbf{w})p(\mathbf{w})d\mathbf{w}$$

- Non è necessaria:
 - Elimina la dipendenza del problema dai parametri del modello $\mathbf{w}$ che invece sono il nostro obiettivo
 - È da ritenersi una costante perché è l'evidenza del data set a prescindere dalla scelta dei parametri

Stima a massima verosimiglianza (MLE)

- La MLE è uno dei principi che possono essere utilizzati per definire se una particolare funzione *rappresenta un buon stimatore per una certa classe di modelli di apprendimento*
- La stima MLE assume che il nostro modello massimizzi la sua probabilità $P(\mathcal{D}, \mathbf{w})$ *solo attraverso la massimizzazione di $P(\mathcal{D} | \mathbf{w})$*
- La stima sarà **campionaria** perché possediamo solo n manifestazioni $\mathbf{x}^{(i)}$ di $\mathbf{x}$, ovvero il nostro data set $\mathcal{D}$.

Stima a massima verosimiglianza (MLE)

$$\mathcal{D} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}, \quad p_{\text{data}}(\mathbf{x}) \quad \text{non nota}$$

Stima a massima verosimiglianza (MLE)

$$\mathcal{D} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}, \quad p_{\text{data}}(\mathbf{x}) \quad \text{non nota}$$

$$\mathbf{w}_{\text{ML}} = \arg \max_{\boldsymbol{\theta}} p_{\text{model}}(\mathcal{D}|\mathbf{w})$$

Stima a massima verosimiglianza (MLE)

$$\mathcal{D} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}, \quad p_{\text{data}}(\mathbf{x}) \quad \text{non nota}$$

$$\mathbf{w}_{\text{ML}} = \arg \max_{\boldsymbol{\theta}} p_{\text{model}}(\mathcal{D}|\mathbf{w})$$

$$= \arg \max_{\mathbf{w}} \prod_{i=1}^m p_{\text{model}}(\mathbf{x}^{(i)}|\mathbf{w})$$

Assunzione iid sui campioni

Stima a massima verosimiglianza (MLE)

$$\mathbf{w}_{\text{ML}} = \arg \max_{\mathbf{w}} \sum_{i=1}^m \log p_{\text{model}} \left(\mathbf{x}^{(i)} | \mathbf{w} \right)$$

Passiamo ai logaritmi: otteniamo una sommatoria

Stima a massima verosimiglianza (MLE)

$$\begin{aligned}\boldsymbol{w}_{\text{ML}} &= \arg \max_{\boldsymbol{w}} \sum_{i=1}^m \log p_{\text{model}} \left(\boldsymbol{x}^{(i)} | \boldsymbol{w} \right) = \\ &= \arg \max_{\boldsymbol{w}} \mathbb{E}_{\boldsymbol{x} \sim \hat{p}_{\text{data}}} \log p_{\text{model}} (\boldsymbol{x} | \boldsymbol{w})\end{aligned}$$

nota empiricamente

dai dati osservati $\rightarrow$ distribuzione empirica

Stima a massima verosimiglianza (MLE)

- Possiamo assumere di voler minimizzare la distanza tra la distribuzione empirica sui dati e quella del modello in termini della loro D_{KL}

*non dipende dai parametri
del modello*

$$D_{KL}(\hat{p}_{\text{data}} || p_{\text{model}}) = \mathbb{E}_{\mathbf{x} \sim \hat{p}_{\text{data}}} [\log \hat{p}_{\text{data}}(\mathbf{x}) - \log p_{\text{model}}(\mathbf{x}|\mathbf{w})]$$

Stima a massima verosimiglianza (MLE)

- Possiamo assumere di voler minimizzare la distanza tra la distribuzione empirica sui dati e quella del modello in termini della loro D_{KL}

*non dipende dai parametri
del modello*

$$\begin{aligned} D_{KL}(\hat{p}_{\text{data}} \parallel p_{\text{model}}) &= \mathbb{E}_{\mathbf{x} \sim \hat{p}_{\text{data}}} [\log \hat{p}_{\text{data}}(\mathbf{x}) - \log p_{\text{model}}(\mathbf{x}|\mathbf{w})] = \\ &= -\mathbb{E}_{\mathbf{x} \sim \hat{p}_{\text{data}}} [\log p_{\text{model}}(\mathbf{x}|\mathbf{w})] = \\ &= -\sum_{\mathbf{x} \sim \hat{p}_{\text{data}}} \hat{p}_{\text{data}}(\mathbf{x}|\mathbf{w}) \log p_{\text{model}}(\mathbf{x}|\mathbf{w}) = -\sum_{i=1}^n \hat{p}_{\text{data}}(\mathbf{x}^{(i)}|\mathbf{w}) \log p_{\text{model}}(\mathbf{x}^{(i)}|\mathbf{w}) \end{aligned}$$

*Cross-entropia della
distribuzione del modello
Rispetto a quella dei dati!!*

Stima a massima verosimiglianza (MLE)

- Si può mostrare che *la minimizzazione del MSE corrisponde alla MLE se la distribuzione dei dati è gaussiana*
- L'ipotesi in questo caso è che il modello formi una stima $\hat{y}(\mathbf{x}, \mathbf{w})$ dai dati *del valor medio della gaussiana, ma non della varianza*

Stima Maximum A Posteriori (MAP)

- La MLE rappresenta l'approccio alla massimizzazione della stima nell'ottica della *statistica frequentista*
- La stima dei parametri $\mathbf{w}$ è una variabile statistica che è funzione dei dati, anch'essi visti come casuali
- L'incertezza sulla stima è rappresentata dalla sua varianza

Stima Maximum A Posteriori (MAP)

- Nella *statistica Bayesiana*, al contrario, i dati sono considerati come già osservati e non sono casuali mentre $\mathbf{w}$ è esatto, ma non noto
- La nostra conoscenza imperfetta su $\mathbf{w}$ è rappresentata dal *prior* $p(\mathbf{w})$ noto prima di osservare i dati
- La scelta del prior è fondamentale perché deve riflettere la nostra conoscenza su $\mathbf{w}$ e deve essere ad elevato contenuto di informazione

Stima Maximum A Posteriori (MAP)

- Da un punto di vista operativo, nel nostro processo di massimizzazione, usando al solito i logaritmi, otteniamo:

$$\mathbf{w}_{\text{MAP}} = \arg \max_{\mathbf{w}} p(\mathbf{w}|\mathbf{x}) = \arg \max_{\mathbf{w}} (\log p(\mathbf{x}|\mathbf{w}) + \log p(\mathbf{w}))$$

- Si tratta della somma della MLE e del prior
- La stima MAP riduce la varianza dello stimatore rispetto a quella MLE, ma in genere ne aumenta il bias

Tipologie dei compiti di apprendimento ($\mathcal{T}$)

- Clustering

- L'algoritmo suddivide il data set in k partizioni modo che ogni partizione (o *cluster*) contenga dati molto più simili tra loro che rispetto agli appartenenti ad altre partizioni

$$f : \mathbb{R}^n \rightarrow \{1, \dots, k\}$$

- Le partizioni *emergono dalla struttura interna dei dati*
- La distribuzione $P(\mathcal{D}, \mathbf{w})$ è multinomiale

Tipologie dei compiti di apprendimento (T)

- Classificazione

- L'algoritmo stima una funzione che fa corrispondere ogni ingresso ad una di k classi distinte

$$f : \mathbb{R}^n \rightarrow \{1, \dots, k\}$$

- La differenza con il clustering è che *le classi rappresentano un processo esogeno sui dati*
- Anche in questo caso la distribuzione $P(\mathcal{D}, \mathbf{w})$ è multinomiale

Tipologie dei compiti di apprendimento (T)

- Regressione
 - Predizione di un valore numerico a partire dagli ingressi
 - Simile alla classificazione, ma la la funzione $f: \mathbb{R}^n \rightarrow \mathbb{R}$
 - La distribuzione $P(\mathcal{D}, \mathbf{w})$ è gaussiana

Tipologie dei compiti di apprendimento (T)

- Rilevamento di anomalie (anomaly detection)
 - Individuazione di ingressi tali da essere eccezionali o atipici rispetto ad un dato comportamento «normale» dei campioni della popolazione
 - Riconducibili a compiti di clustering e/o classificazione interpretati in forma duale e cioè cercando gli elementi «non appartenenti» alla classe normale

Tipologie dei compiti di apprendimento (T)

- Trascrizione
 - OCR, speech recognition
 - E' riconducibile alla classificazione
- Traduzione automatica (machine translation)
 - Conversione di una sequenza discreta di simboli in un'altra
 - Applicata al linguaggio naturale

Tipologie dei compiti di apprendimento (T)

- Traduzione automatica (machine translation)
 - Conversione di una sequenza discreta di simboli in un'altra
 - Applicata al linguaggio naturale

Tipologie dei compiti di apprendimento (T)

- Compiti con output strutturato
 - Task in cui l'uscita è un vettore o un'altra struttura dati i cui valori sono strettamente correlati tra loro
 - Task di NLP quali il parsing o il QA
 - Segmentazione e annotazione di immagini
- } *Sono ancora delle classificazioni*

Tipologie dei compiti di apprendimento (T)

- Sintesi e campionamento
 - Generazione di campioni simili ai dati di ingresso
 - Applicazioni in campo multimediale
 - Generazione automatica di texture nei video-game
 - Speech synthesis

Tipologie dei compiti di apprendimento ($\mathcal{T}$)

- Imputazione di dati mancanti
 - Predizione di feature mancanti a partire da input incompleto
- Denoising
 - Predizione di $\mathbf{x} \in \mathbb{R}^n$, noto che sia $\mathbf{x}'$ versione corrotta di $\mathbf{x}$
 - Più in generale, predizione di $p(\mathbf{x} \mid \mathbf{x}')$

Utilizzo dei dati (E)

- Apprendimento non supervisionato (unsupervised learning)
 - In questi algoritmi si deve apprendere la distribuzione di probabilità $p(\mathbf{x})$ che sottende il data set
 - Si è interessati, in generale, ad apprendere la struttura dei dati
 - Es. clustering, compiti di sintesi, denoising

Utilizzo dei dati

- Apprendimento supervisionato (supervised learning)
 - In questi algoritmi ogni campione $\mathbf{x}$ dei dati di ingresso è associato ad una etichetta $\mathbf{y}$ numerica o testuale
 - L'algoritmo deve apprendere a predire $\mathbf{y}$ dato $\mathbf{x}$, ovvero a predire $p(\mathbf{y} \mid \mathbf{x})$
 - Classificazione e regressione, anche con reti neurali

Utilizzo dei dati (E)

- Apprendimento per rinforzo (reinforcement learning)
 - Il data set non è fisso e l'algoritmo interagisce con l'ambiente, apprendendo tramite una procedura di prova ed errore
 - L'algoritmo (o meglio l'agente) al tempo t si trova in uno stato S_t ed esegue un'azione A_t ottenendo una ricompensa o «reward» R_t
 - L'agente si evolve lungo un ciclo stato-azione-ricompensa

Utilizzo dei dati (E)

- Apprendimento per rinforzo (reinforcement learning)
 - La ricompensa globale G_t al tempo t è la somma di tutti i possibili valori di ricompensa legati ai comportamenti futuri, almeno fino ad un certo orizzonte temporale di T istanti di tempo nel futuro.

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \text{ where } \gamma \in [0, 1)$$

- γ è il «fattore di sconto» che pesa sempre meno le azioni compiute nel futuro più lontano

Utilizzo dei dati (E)

- Tipi di apprendimento per rinforzo: uso di una policy
 - L'agente utilizza una *funzione di policy* π per la scelta dell'azione a rispetto allo stato corrente s : $a = \pi(s)$
 - a) L'agente cerca di massimizzare il valore di ricompensa, come valore atteso rispetto alla policy:
$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$
 - b) L'agente cerca di massimizzare direttamente la funzione π eventualmente definita in forma probabilistica come $\pi(a \mid s) = P[A_t = a \mid S_t = s]$

Utilizzo dei dati (E)

- Tipi di apprendimento per rinforzo: Q learning
 - Si addestra un modello di ML/DL per massimizzare una funzione di «qualità» $Q(s_t, a_t)$ ottenuta se all'istante t si intraprende una data azione a_t nello stato corrente s_t :

$$Q(s_t, a_t) \leftarrow \underbrace{Q(s_t, a_t)}_{\text{vecchio valore}} + \underbrace{\alpha_t(s_t, a_t)}_{\text{tasso di apprendimento}} \times \left[\underbrace{R_{t+1}}_{\text{ricompensa}} + \underbrace{\gamma}_{\text{fattore di sconto}} \underbrace{\max_{a_{t+1}} Q(s_{t+1}, a_{t+1})}_{\text{valore futuro massimo}} - \underbrace{Q(s_t, a_t)}_{\text{vecchio valore}} \right]$$
$$Q(s_t, a_t) \leftarrow Q(s_t, a_t)(1 - \alpha_t(s_t, a_t)) + \alpha_t(s_t, a_t)[R_{t+1} + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1})]$$

Capacità e generalizzazione

- L'obiettivo principale di un algoritmo di apprendimento è quello di *generalizzare* a partire dai dati osservati, cioè *fare corrette predizioni su dati che non ha mai osservato prima*
- Per raggiungere l'obiettivo, ogni algoritmo usa un'apposita misura di performance P volta a *minimizzare l'errore commesso nella predizione*

Capacità e generalizzazione

- Ogni algoritmo deve minimizzare due misure di errore:
 - Training error, cioè la misura di errore sui dati usati per l'addestramento
 - Test error o generalization error, cioè la misura di errore sui dati utilizzati per testare la bontà della performance e che *sono assolutamente distinti da quelli di addestramento*
- La forma funzionale L di queste misure di errore, che in generale chiameremo *loss*, è la stessa e dipende, algoritmo per algoritmo, *dal tipo di stima statistica che stiamo conducendo*

Capacità e generalizzazione

- La capacità di generalizzare si basa sulla Teoria dell'Apprendimento Statistico
 - Esiste un processo statistico unico che descrive il fenomeno sotto esame e che genera non solo i dati usati per l'addestramento, ma anche quelli mai visti
 - Questo processo è descritto da un'unica distribuzione di probabilità p_{data} , non nota a priori
 - Si stabilisce un'ipotesi a priori sui dati che è il fondamento di ogni algoritmo di ML: i dati di addestramento e di test sono *indipendenti e identicamente distribuiti (independent and identically distributed - i.i.d.)*

Capacità e generalizzazione

- L'ipotesi i.i.d. ci dice che, per qualunque modello, il valore atteso del training error e del test error sono uguali
 - Esiste certamente una configurazione dei parametri $\mathbf{w}$ per cui questi due errori sono uguali
 - Nella realtà, a causa del campionamento dei dati disponibili, questa condizione non si verifica

Capacità e generalizzazione

- L'ipotesi i.i.d. ci dice che, per qualunque modello, il valore atteso del training error e del test error sono uguali
 - L'algoritmo cerca di minimizzare il training error attraverso l'addestramento sul training set che è una procedura di minimizzazione della loss
 - L'algoritmo cerca di mantenere minimo il gap con il generalization error

Capacità e generalizzazione

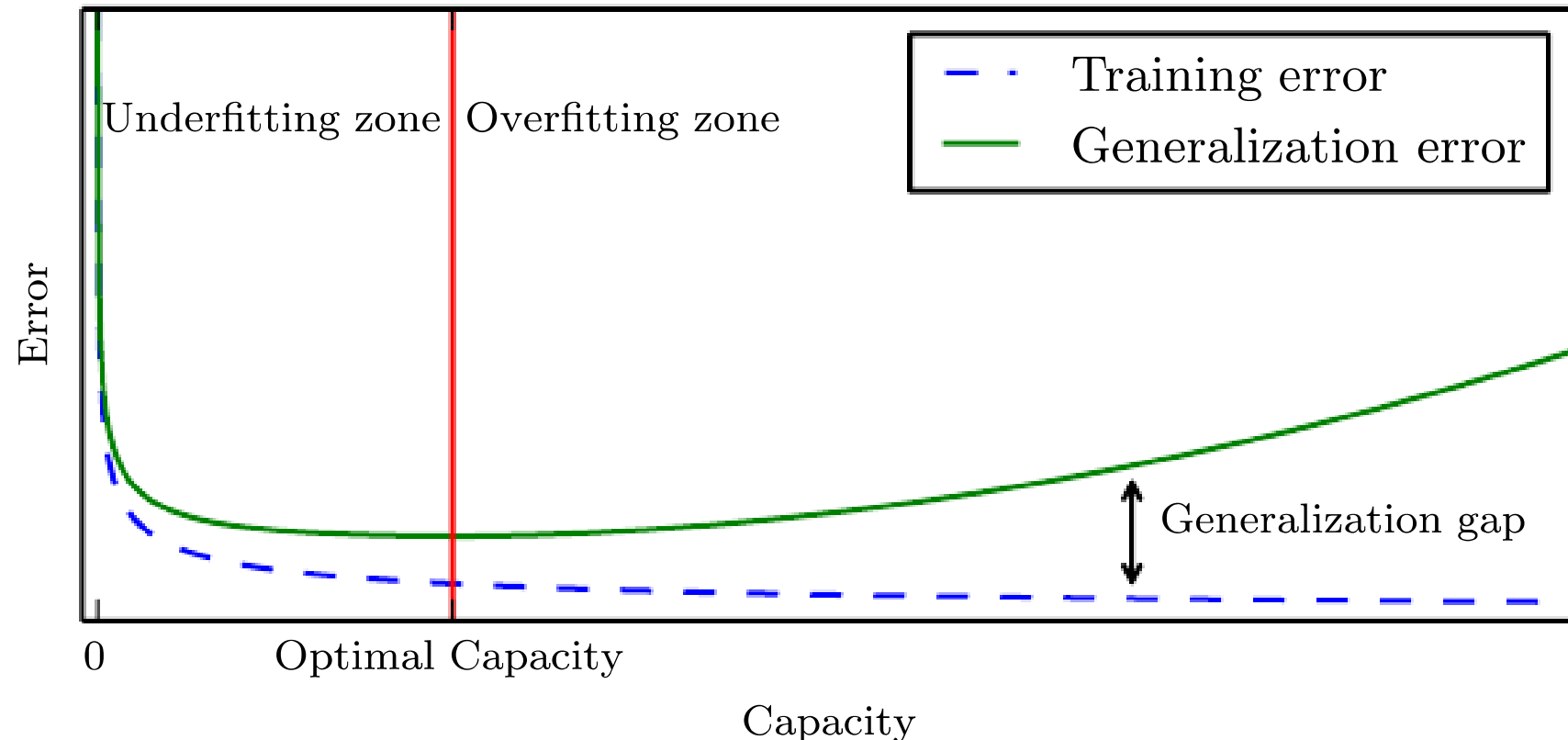
- Overfitting
 - Eccesso di minimizzazione del training error, con conseguente scarsa generalizzazione (generalization error elevato)
- Underfitting
 - Bassa abilità di minimizzare il training error

Capacità e generalizzazione

- Si parla di *capacità* di un modello con riferimento al fatto che questo riesca ad approssimare la gamma di forme funzionali più vasta possibile
- Il ventaglio di forme funzionali direttamente ottenibili dal nostro modello si dirà «spazio delle ipotesi»
- La capacità del modello sarà *gestibile tramite i suoi iperparametri*

Capacità e generalizzazione

- Si parla di *capacità* di un modello con riferimento al fatto che questo riesca ad approssimare la gamma di forme funzionali più vasta



Capacità e generalizzazione

- «No free lunch» theorem
 - La performance di un qualunque algoritmo di apprendimento, mediata su qualunque distribuzione di probabilità che genera i dati, *è la stessa*
 - Non ci sono algoritmi in linea di principio migliori degli altri
 - In realtà il disegno di un modello basato su particolari assunzioni fatte su p_{data} , ovvero la scelta accurata dello spazio delle ipotesi, rende certi algoritmi migliori degli altri su quella particolare forma di processo di generazione dei dati

Capacità e generalizzazione

- Esempio – la regressione

$$y = \mathbf{w} \cdot \mathbf{x} + b, \quad \text{regressione lineare}$$

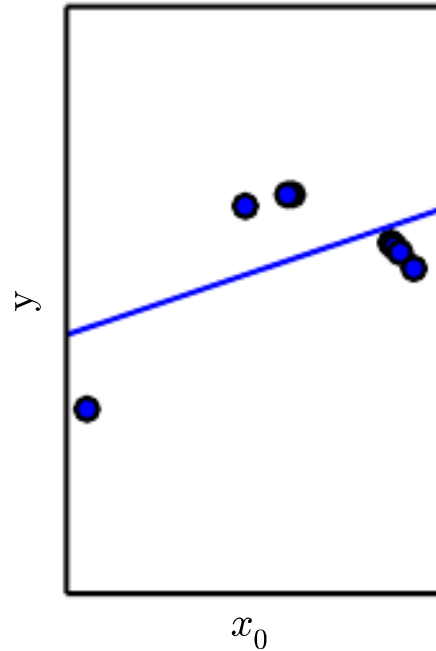
$$y = \sum_{i=1}^n \mathbf{w}_i \cdot \mathbf{x}^i + b, \quad \text{regressione polinomiale}$$

- Spazio delle ipotesi $\rightarrow$ funzioni polinomiali
- Parametri $\rightarrow \{b, \mathbf{w}_i, i = 1, \dots, n\}$
- Iperparametri $\rightarrow n$

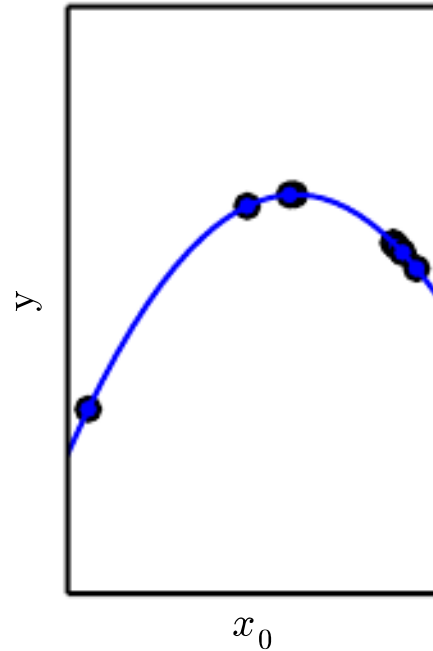
Capacità e generalizzazione

- Esempio – la regressione

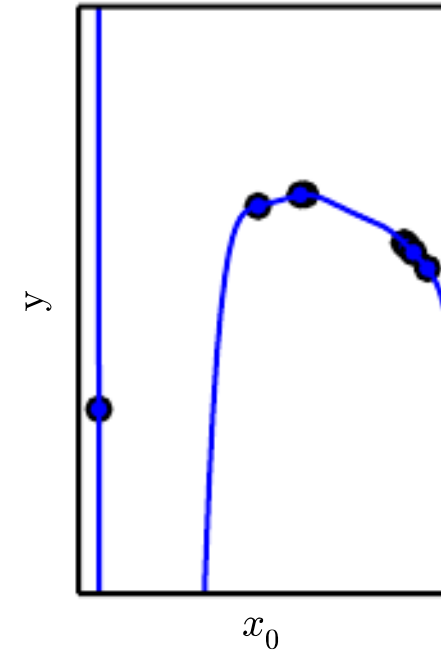
Underfitting



Appropriate capacity



Overfitting

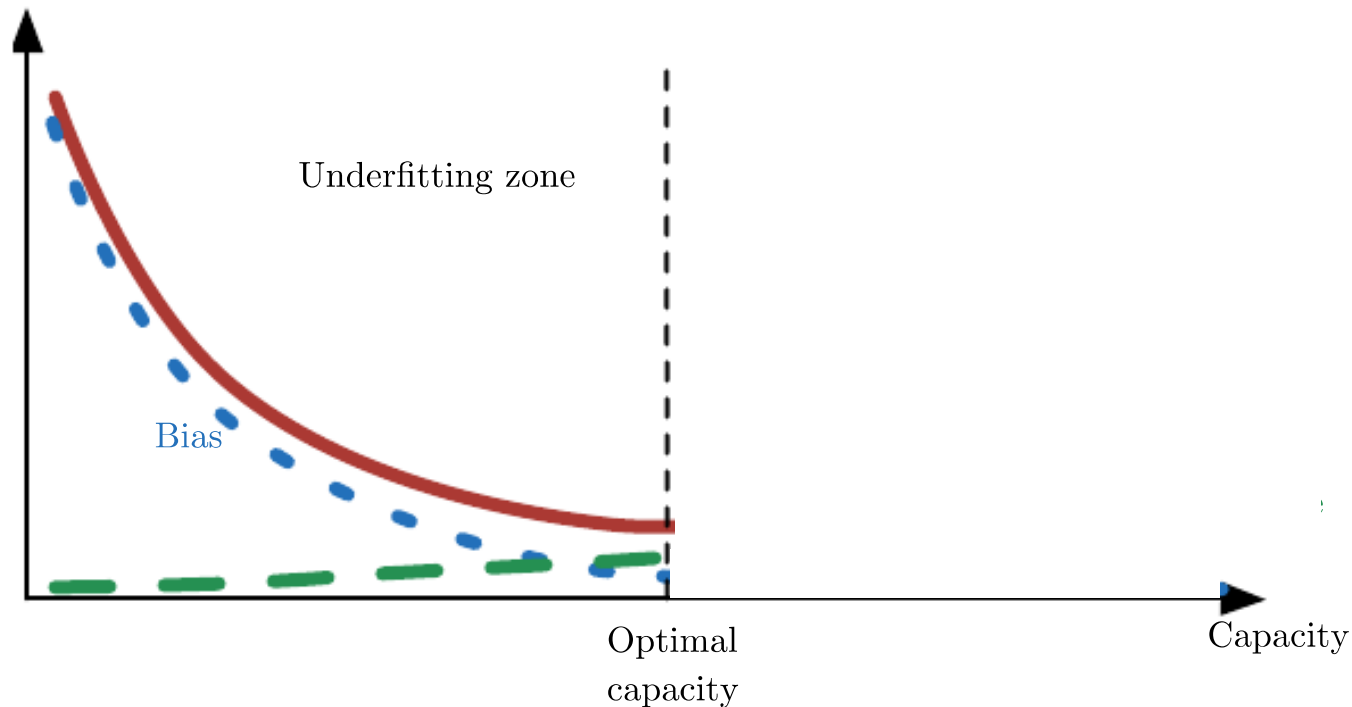


Capacità e generalizzazione

- Esempio – la regressione
 - L'algoritmo di regressione apprende minimizzando il MSE_{train} calcolato sul training set, mantenendo minimo il gap dal generalization error MSE_{test}
 - Come ricordiamo, ciò corrisponde ad una stima MLE dei parametri ottimi di $p_{model}(y|\mathbf{x}_{\sim p_{data}})$ supposta Gaussiana
 - Analizziamo la bontà della stima in termini della capacità

Capacità e generalizzazione

- Esempio – la regressione



$$\text{MSE} = \mathbb{E} \left[(\hat{\theta}_m - \theta)^2 \right] =$$

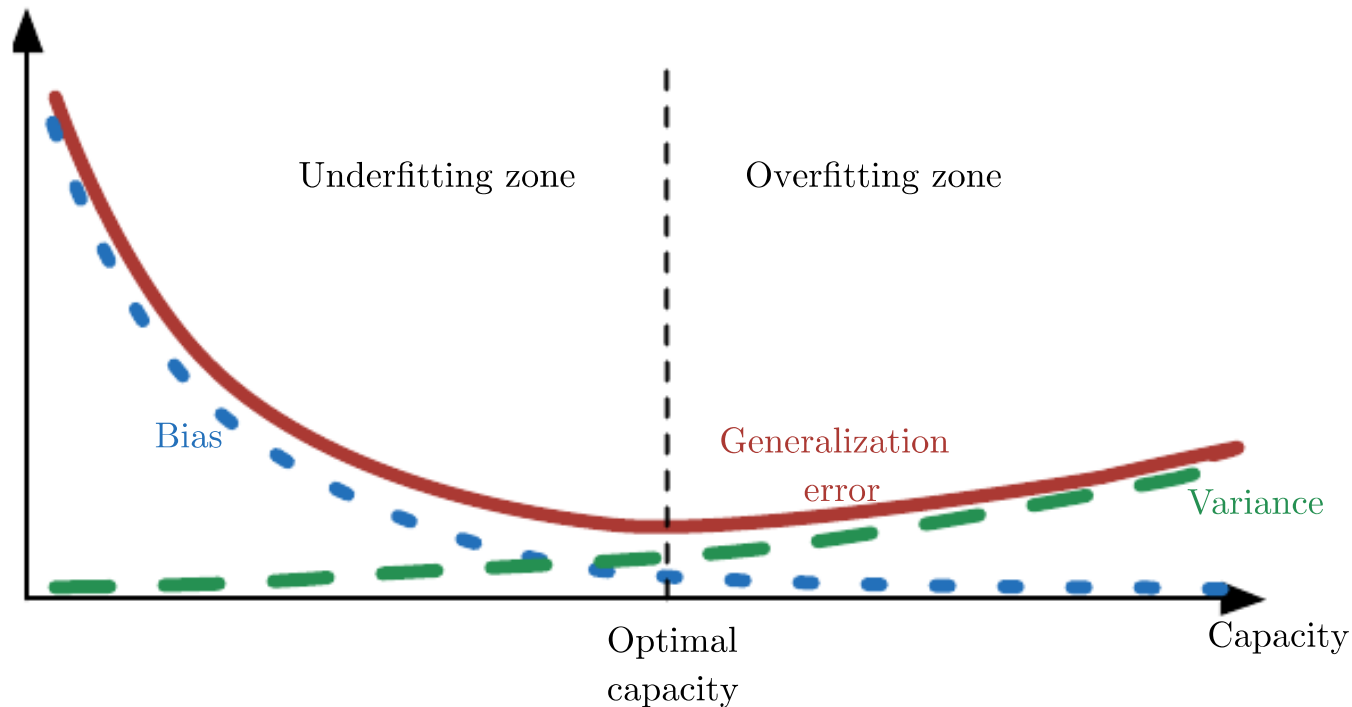
$$\text{Bias}(\hat{\theta}_m)^2 + \text{Var}(\hat{\theta}_m)$$

Underfitting:

- il training set è appreso sempre meglio al crescere della capacità
- il bias (valore atteso sul training set) decresce
- la varianza (dovuta al test set) non cresce di molto

Capacità e generalizzazione

- Esempio – la regressione



$$\text{MSE} = \mathbb{E} \left[(\hat{\theta}_m - \theta)^2 \right] =$$

$$\text{Bias}(\hat{\theta}_m)^2 + \text{Var}(\hat{\theta}_m)$$

Overfitting:

- il training set è appreso sempre meglio al crescere della capacità
- il bias sul training set diventa minimo
- la varianza (dovuta al test set) diviene molto elevata

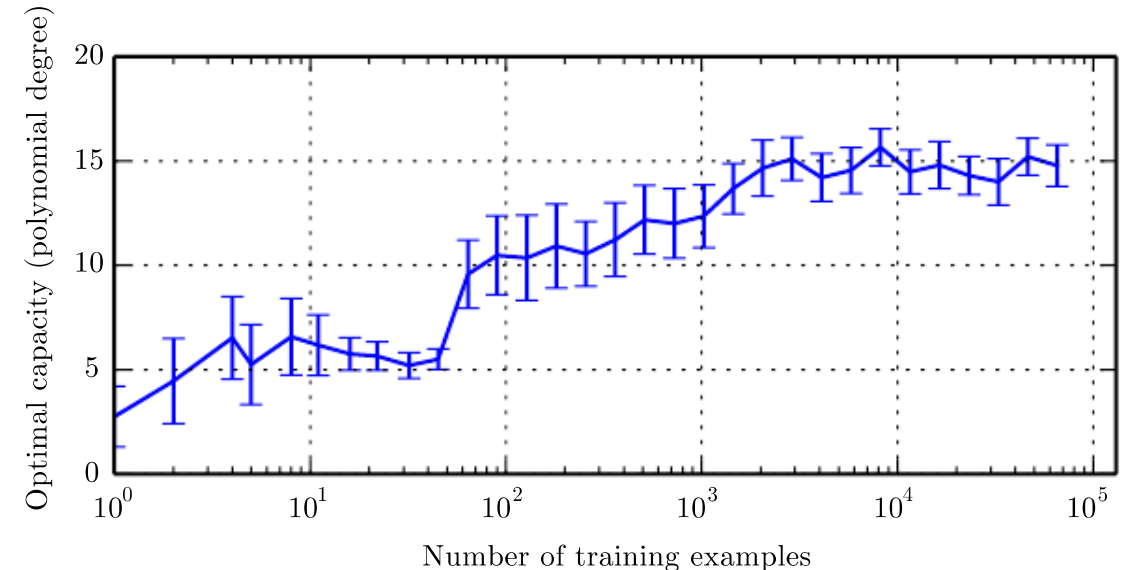
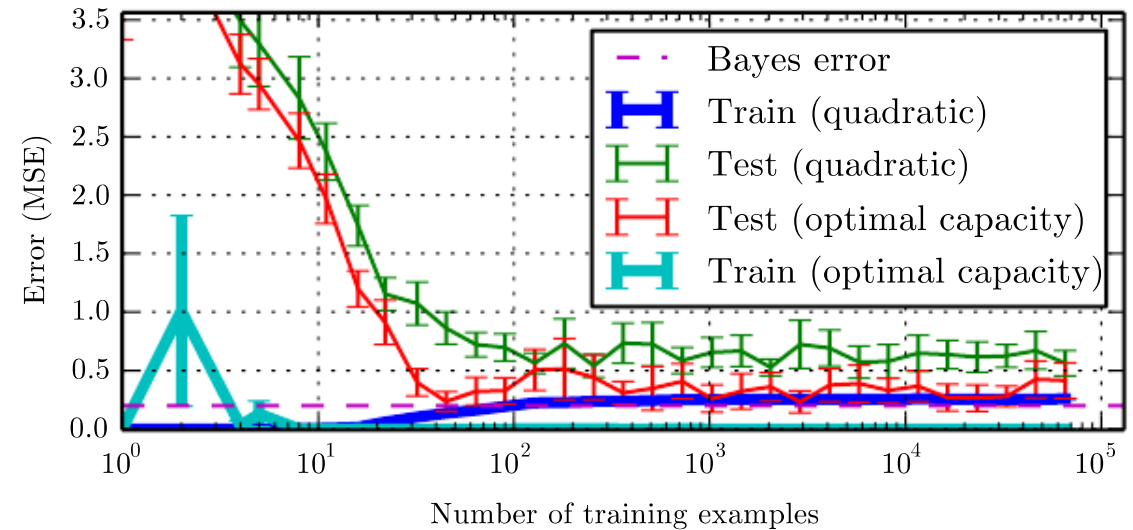
Capacità e generalizzazione

- Il training error e la generalizzazione di un algoritmo di apprendimento dipendono anche dalle dimensioni del training set
- In linea di principio un «oracolo» che conosca esattamente p_{data} commette un errore costante di predizione noto come *Bayes error*
- L'oracolo non conosce «esattamente» i dati, ma la loro distribuzione di probabilità

Capacità e generalizzazione

- Al crescere dei dati di addestramento, L'errore di generalizzazione tende asintoticamente ad un valore che è maggiore (in caso di bassa capacità del modello) o uguale al Bayes error

Data set sintetico generato aggiungendo rumore ai punti ottenuti da un polinomio di grado 5



Tecniche di addestramento

- Gli algoritmi di apprendimento utilizzano diverse tecniche che modificano le procedure di addestramento standard al fine di aumentare la *capacità effettiva* del modello prescelto
- Queste tecniche impattano direttamente sulla scelta degli iperparametri da cui dipende la *capacità di rappresentazione* (teorica) del modello
 - Regularizzazione
 - Validation set e cross-validation

Tecniche di addestramento

- Regularizzazione
 - Una qualunque modifica dell'algoritmo di apprendimento *mirata a ridurre esplicitamente l'errore di generalizzazione*, ma non il training error
 - Il termine di regularizzazione ci consente di scegliere alcune forme funzionali rispetto alle altre nel nostro spazio delle ipotesi

Tecniche di addestramento

- Regularizzazione

- Esempio – regularizzazione «weight decay» per la regressione che esprime preferenza per piccoli $\mathbf{w}$

Underfitting
(Excessive λ)

Appropriate weight decay
(Medium λ)

Overfitting
($\lambda \rightarrow 0$)

λ è un ulteriore iperparametro

$$\mathcal{L}(\mathbf{w}) = \text{MSE}_{\text{train}} + \lambda \mathbf{w}^\top \mathbf{w}$$

Un tipico effetto della regularizzazione è lo *smoothing* della funzione di loss per cui, analiticamente, $\mathcal{L}$ converge verso un *minimo globale diverso* dalla pura MSE

x_0

x_0

x_0

Il modello è un polinomio di grado 9

Tecniche di addestramento

- Validation set e cross-validation
 - Il validation set si utilizza per ovviare al problema dell'*apprendimento degli iperparametri*
 - Il validation set è una porzione del training set che viene espunta da quest'ultimo prima di iniziare l'addestramento vero e proprio

Tecniche di addestramento

- Validation set e cross-validation
 - Il validation set viene utilizzato per verificare l'andamento del generalization error, modificando gli iperparametri per minimizzarlo
 - L'errore commesso sul validation set durante l'addestramento risulta essere una stima più corretta dell'errore di generalizzazione rispetto al training error

Tecniche di addestramento

- La procedura di training restituisce il miglior modello rispetto al validation set

Algorithm 1 Full training procedure

Input: $A, \mathbf{x}^{(train)}, split_perc, H$ { A : the algorithm, $\mathbf{x}^{(train)}$: the training data, $split_perc$: split percentage, H : hyperparameters search space}

$\mathbf{w} \leftarrow \text{init_weights}()$ {initialize the parameters
as either zero or random values}

$L \leftarrow \infty, L^{val} \leftarrow \infty$ {initial values of the loss both for training
and validation}

$(train_set, val_set) \leftarrow \text{split}(\mathbf{x}^{(train)}, split_perc)$ {split the training data}

for all $\theta \in H$ **do**

$(L, \mathbf{w}) \leftarrow A(train_set, \mathbf{w}, \theta)$ {training provides loss and weights}

if $L(val_set, \mathbf{w}, \theta) < L^{val}$ **then**
{save the state of the model related to the best validation error}

$L^{val} \leftarrow L(val_set, \mathbf{w}, \theta)$

$\theta^* \leftarrow \theta$

$\mathbf{w}^* \leftarrow \mathbf{w}$

$L^* \leftarrow L$

end if

end for

Output: $L^*, \mathbf{w}^*, \theta^*$ {output the best model as regards validation}

Tecniche di addestramento

- Validation set e cross-validation
 - Quando ci si trova con un numero ridotto di campioni si adotta la tecnica della *k-fold cross validation*
 - Il data set viene suddiviso in k partizioni che non si sovrappongono
 - Per ogni partizione si adotta una strategia di addestramento per cui la partizione fa da test set e il resto da training set

Tecniche di addestramento

- Validation set e cross-validation
 - All'interno di ogni run di training si può individuare un validation set per addestrare il modello rispetto ai propri iperparametri
 - L'errore commesso viene calcolato come il *valore medio su tutte le partizioni* e riportato come performance del modello sull'intero data set
 - E' possibile anche *inserire la cross-validation nel processo di ricerca dei migliori iperparametri del modello*

Tecniche di addestramento

Define $\text{KFoldXV}(\mathbb{D}, A, L, k)$:

Require: $\mathbb{D}$, the given dataset, with elements $\mathbf{z}^{(i)}$

Require: A , the learning algorithm, seen as a function that takes a dataset as input and outputs a learned function

Require: L , the loss function, seen as a function from a learned function f and an example $\mathbf{z}^{(i)} \in \mathbb{D}$ to a scalar $\in \mathbb{R}$

Require: k , the number of folds

Split $\mathbb{D}$ into k mutually exclusive subsets $\mathbb{D}_i$, whose union is $\mathbb{D}$

for i from 1 to k **do**

$f_i = A(\mathbb{D} \setminus \mathbb{D}_i)$

for $\mathbf{z}^{(j)}$ in $\mathbb{D}_i$ **do**

$e_j = L(f_i, \mathbf{z}^{(j)})$

end for

end for

Return e